A Decentralized Voting System

Fengxi Suchecki-Zhao

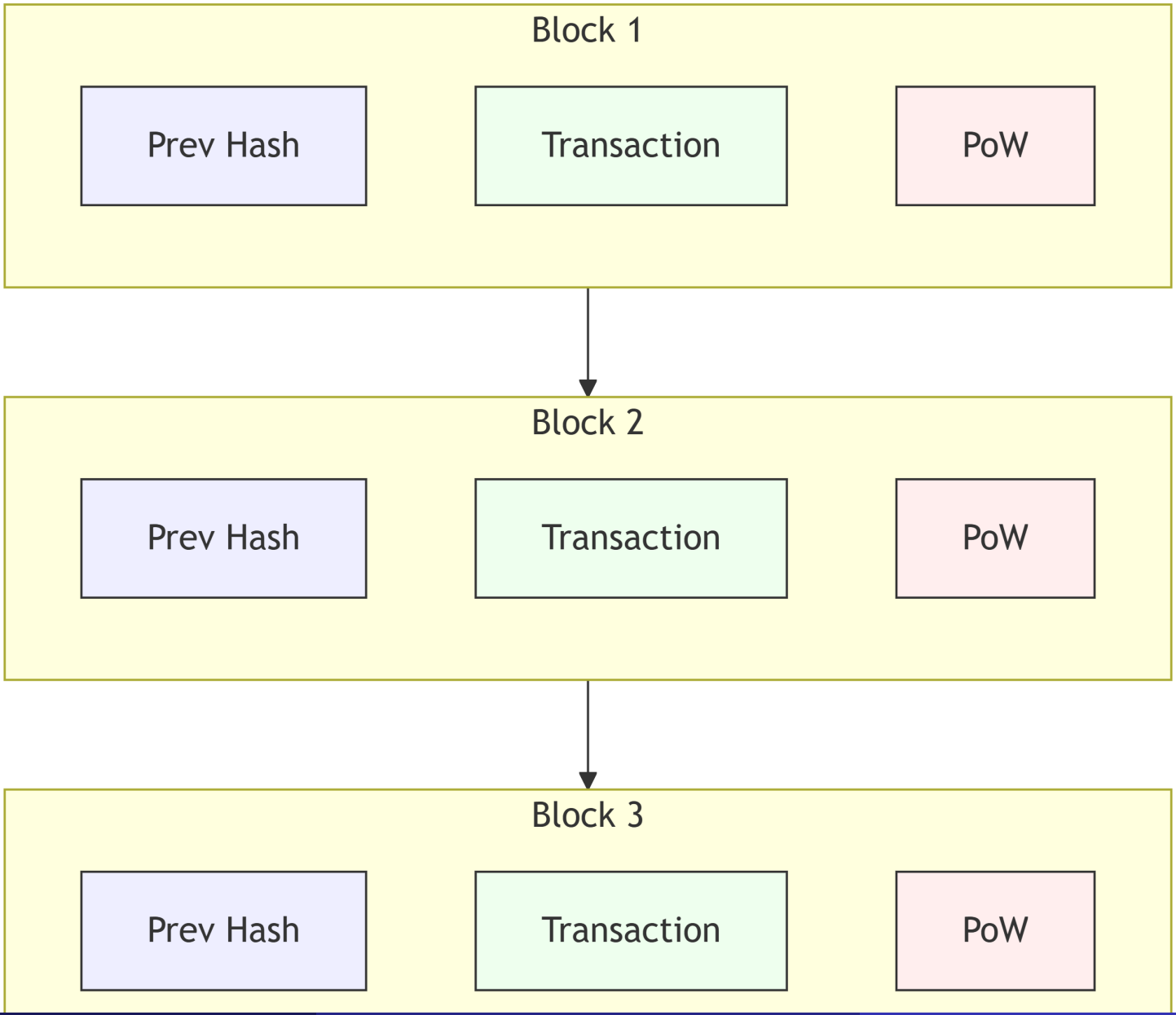
February 20, 2026

Outline

Motivation

- **Traditional paper elections** are costly and slow: logistics, staffing, physical security, and recounts create delays and opportunities for human error.
- **Centralized online voting** improves convenience but concentrates trust:
  - **single point of failure** (outages, misconfiguration),
  - **insider risk** (administrators),
  - limited **independent auditability**.
- A decentralized approach aims for **public verifiability** without a trusted back-end, while preserving privacy via **commit–reveal**.

Blockchain Background



Blockchain Background

- A **transaction** invokes contract code and updates on-chain state.
- Confirmed blocks make the history **tamper-evident** and widely replicated.
- **Smart contracts** are public and deterministic:
  - code and state are readable by anyone,
  - execution can be replayed to verify outcomes.
  - **rule enforcement**.
  - **no trusted server**.

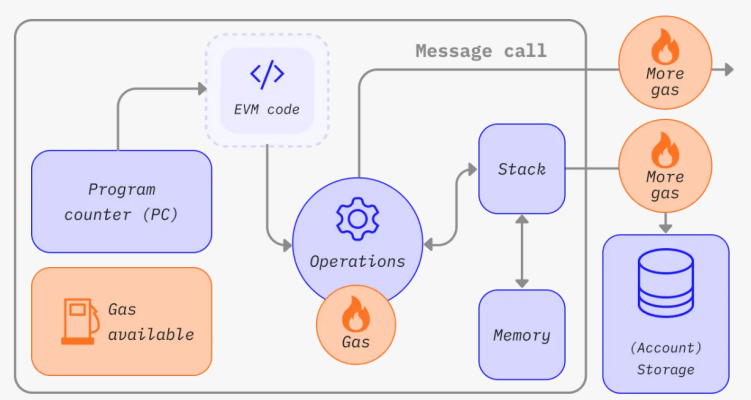


Figure: Ethereum Virtual Machine (EVM) [4].

Lifecycle Overview

- One contract instance progresses through a fixed sequence of phases.
- Phases are **one-way**: **Initialization** $\rightarrow$ **Registration** $\rightarrow$ **Commit** $\rightarrow$ **Reveal** $\rightarrow$ **Finalized**.
- Every state-changing method is protected by **phase gating**: calls made in the wrong phase revert, so the workflow is enforced on-chain.

Phases

- **Initialization:** chair deploys/configures the election instance.
- **Registration:** chair registers eligible voters and finalizes the proposal set. Chair sets commit/reveal deadlines.
- **Commit:** each registered voter submits a *commitment hash*.
- **Reveal:** voters reveal *choice* + *salt*. The contract verifies the commitment and counts the vote.
- **Finalized:** results become readable as the final tally.

Lifecycle Overview

Phase changes

- **Registration → Commit:** chair closes registration after voter list and proposals are finalized.
- **Commit → Reveal:** chair advances when **all registered voters committed or** the **commit deadline** has passed.
- **Reveal → Finalized:** chair advances when **all commitments are revealed or** the **reveal deadline** has passed.
- Phase changes and vote actions emit events, enabling off-chain monitoring.

Deadlines

- Deadlines provide **liveness**: the election can finish even if some voters do not participate.
- They also define the **privacy window**: vote choices remain hidden until reveals begin.

Commit–Reveal

Commit (hides the choice)

- Voter picks a proposal index and a random **salt**.
- Voter submits only a **commitment hash** to the contract.
- On-chain data do not reveal the vote choice during the commit phase.

Reveal (verifies and counts)

- Voter reveals **proposal index** + **salt**.
- Contract recomputes the hash and checks it matches the stored commitment.
- If valid, the vote is counted.

Commitment

- The contract computes commitments as:

$$c = \texttt{keccak256(}\\ \texttt{abi.encode(index, salt,}\\ \texttt{voter, contract, chainId))}$$

- Including `voter`, `contract`, and `chainId` prevents replay across accounts, deployments, or networks.
- Losing the salt makes reveal impossible, so the UI must persist the salt securely.

Reveal

- Reveal is accepted only if:
  - the voter is registered,
  - the voter has committed and has not revealed yet,
  - the reveal deadline has not passed,
  - recomputed hash matches the stored commitment.
- On success, the contract increments the chosen proposal's vote counter and emits events for auditing.
- Final results are readable only in *Finalized* (after the reveal window ends).

Auditing

Anyone can read the proposal vote counts and verify that each counted reveal corresponds to a valid commitment.

Architecture: components and interaction paths

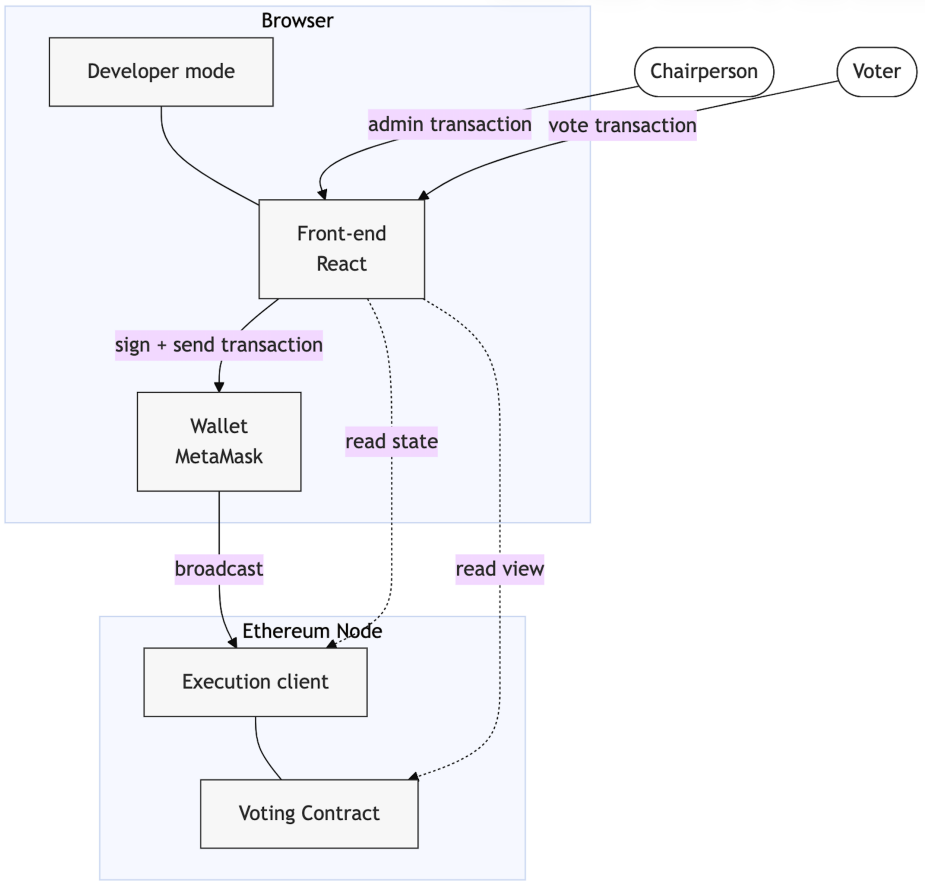


Figure: High-level architecture and interaction paths [1].

Roles

- **Chairperson:** election setup and phase progression
- **Voters:** commit and reveal ballots
- **Observers:** audit events and recompute tally

Trust boundaries

- The contract is the **source of truth**.
- Any party can verify the final state from on-chain data.

Smart Contract

- **Phase-gated workflow:** every state-changing method reverts unless called in the correct phase.
- **Role restriction:** chair-only administration (registration, proposal finalization, deadlines, phase transitions).
- **Commit–reveal correctness:**
  - enforces one commit and one reveal,
  - verifies `hash(choice, salt)` against the stored commitment before counting.
- **Liveness**.
- **Auditability.**

Front-end

- The front-end continuously queries on-chain state (phase, deadlines, voter status) and only exposes valid actions.
- The wallet signs transactions.
- The front-end generates and persist a random salt locally. Submit only the commitment hash on-chain.
- Th front-end retrieves the saved salt, submit *choice* + *salt*, and display on-chain verification result.
- The front-end is **not trusted**: any third party can reproduce outcomes from contract state and emitted events.

Front-end State Model

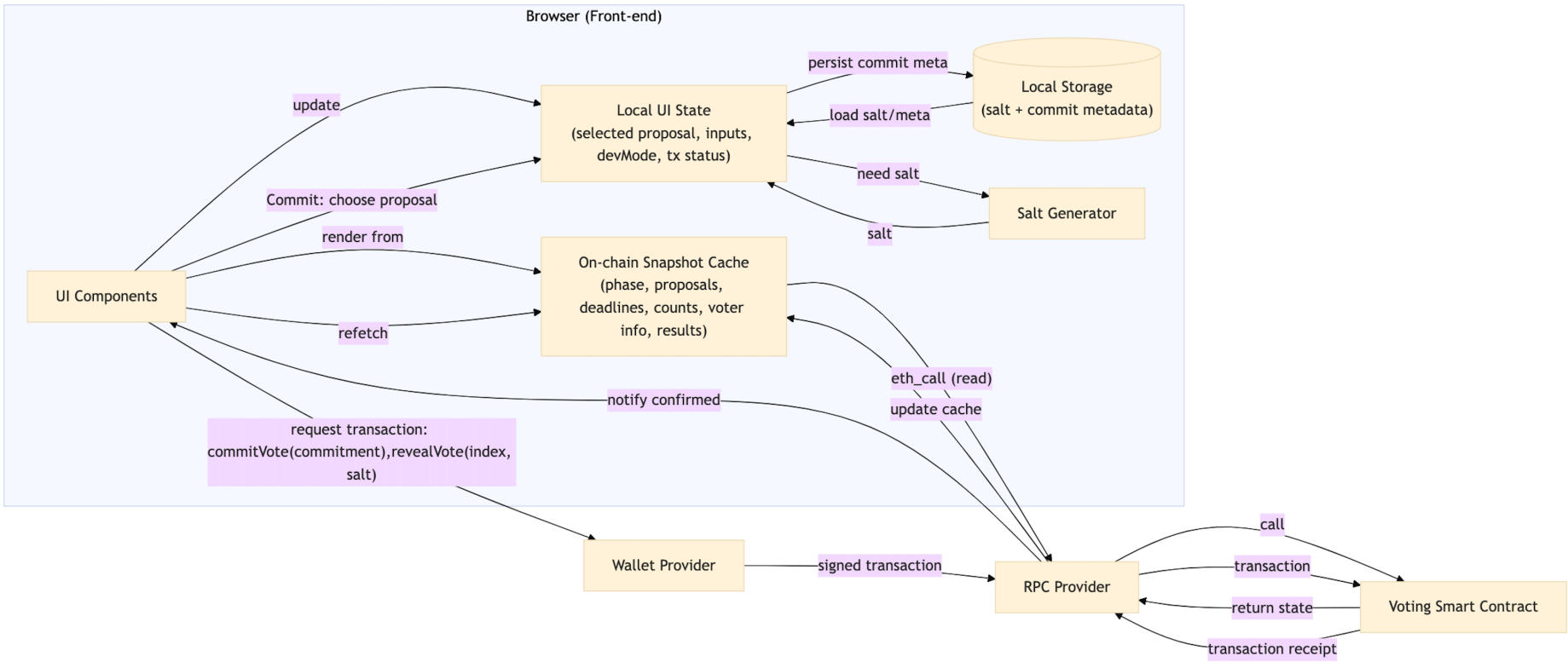


Figure: Front-end state model driven by contract phase and local UI state [1].

Testing

- Automated tests in Hardhat and Mocha[6, 7]:
  - **Phase & role matrix:** each function is tested across all phases and roles.
  - **Negative cases:** wrong phase, non-chair calls, double commit/reveal, invalid reveal, expired deadlines.
  - **Event assertions:** expected events are emitted on success paths.
- Invariants checks after transitions:
  - `revealedCount` $\leq$ `committedCount` $\leq$ `registeredCount`
  - sum of proposal vote counts equals `revealedCount`

Limitations & future work

- **Privacy limits:** addresses and timing remain observable. Commit–reveal hides only until reveal.
- **Liveness:** voters who fail to reveal before the deadline are not counted.
- **Costs:** a lot of gas. Large elections may require Layer-2 ethereum networks.
- **Future work:**
  - stronger privacy (e.g., zero-knowledge),
  - improve UI.

Thank you.

References

Suchecki-Zhao, F. (2026). *A Decentralized Voting System* (Engineering Thesis, Warsaw University of Technology). Figures reused: system architecture, workflow, front-end state model.

Nakamoto, S. (2008). *Bitcoin: A Peer-to-Peer Electronic Cash System*. https://bitcoin.org/bitcoin.pdf

Wood, G. (2014). *Ethereum: A Secure Decentralised Generalised Transaction Ledger (Yellow Paper)*. https://ethereum.github.io/yellowpaper/paper.pdf

Ethereum Foundation. (n.d.). *Ethereum Virtual Machine (EVM) — Official Documentation*. https://ethereum.org/en/developers/docs/evm/

Solidity Documentation. (n.d.). *ABI Encoding and Hashing (keccak256, abi.encode)*. https://docs.soliditylang.org/

Hardhat. (n.d.). *Hardhat Documentation*. https://hardhat.org/

Mocha. (n.d.). *Mocha — JavaScript Test Framework Documentation*. https://mochajs.org/